

05 · Constraint Verification Engine

2026-07-31

Contents

05 · Constraint Verification Engine1

1. 设计哲学：LLM 提假设，Verifier 判真假1

1.1 三态输出2

2. 验证器体系2

2.1 统一接口2

2.2 验证器分派3

3. 自适应容差模型 (Tolerance Model)3

3.1 三态判定规则3

3.2 各关系容差默认值3

3.3 尺度归一化4

4. 验证示例4

4.1 垂直验证 ($OA \perp AB$)4

4.2 切线验证 (AB 切圆 O)4

4.3 椭圆焦点验证 (P 在椭圆上)4

4.4 两圆关系验证4

5. 验证日志与可解释性5

6. 闭环协议：Verifier $\leftrightarrow$ LLM $\leftrightarrow$ Solver5

6.1 工具调用接口5

7. 验证器的正确性保障5

8. 性能优化6

9. 设计路线小结6

05 · Constraint Verification Engine

本文档阐述约束验证引擎（Constraint Verifier）的设计哲学、验证器体系、自适应容差模型、验证日志与闭环协议。Verifier 是阻断”视觉误判 $\rightarrow$ 推理错误”级联的核心机制。

1. 设计哲学：LLM 提假设，Verifier 判真假

本系统刻意将”假设生成”与”真伪判定”解耦，借鉴”生成-判别”范式：

- **LLM / Agent**: 负责提出候选关系与证明思路（创造性、模糊）。
- **Verifier**: 负责用数学方法严格判定（确定性、精确）。

任何 LLM/Agent 提出的几何断言，必须经 Verifier 确认为 true 才能进入证明链。这一设计从根本上抑制了 LLM 的幻觉

问题在几何证明中的危害。

1.1 三态输出

Verifier 不只返回真/假，而是三态：

状态	含义	处理
true	测量值在容差内	入图，可进入证明链
false	测量值远超容差 ($>3 \cdot \text{tol}$)	丢弃，反馈 LLM 反思
uncertain	处于灰区 ($\text{tol} \sim 3 \cdot \text{tol}$)	标记弱假设，LLM 可作为线索但不作为确定前提

三态设计避免硬阈值导致的”一刀切”误判，为推理保留弹性。

2. 验证器体系

验证器	输入	方法	输出
OnVerifier	点 P, 对象 obj	解析距离/参数	true/false/uncertain
CollinearVerifier	点集	最小二乘拟合残差	true/false
AngleVerifier	两线	方向向量夹角	角度值 + 是否 $\approx$ 目标
PerpendicularVerifier	两线	$\cos\theta \approx 0$	true/false
ParallelVerifier	两线	$\sin\theta \approx 0$	true/false
TangentVerifier(line,circle)	直线, 圆	$d \approx r$	true + 切点
TangentVerifier(circle,circle)	两圆	$d \approx r_1 \pm r_2$	true + 切点
OnCircleVerifier	点, 圆	$ \text{OP}-r < \text{tol}$	true/false
EllipseSumVerifier	点 P, 椭圆	$ \text{PF1} + \text{PF2} \approx 2a$	true/false
EqualVerifier	两线段/两角	相对误差	true/false
IntersectionVerifier	两对象	解析求交存在性	交点列表
InscribedVerifier	多边形, 圆	顶点全在圆上	true/false
ConcentricVerifier	两圆	圆心距 $< \text{tol}$	true/false

2.1 统一接口

```
class Verifier(Protocol):
    rel: RelType
    def verify(self, src: Node, dst: Node, attrs: dict) -> VerifyResult:
        ...

class VerifyResult(BaseModel):
    verified: Literal["true","false","uncertain"]
    evidence: str          # 可读证据
    measured: dict = {}    # 测量值 (angle, distance, ...)
    attrs: dict = {}       # 回填属性 (tangent_point 等)
```

2.2 验证器分派

```
VERIFIERS = {
    "On": OnVerifier(),
    "Collinear": CollinearVerifier(),
    "Perpendicular": PerpendicularVerifier(),
    "Parallel": ParallelVerifier(),
    "Tangent": TangentVerifier(), # 内部分 line/circle 与 circle/circle
    "Equal": EqualVerifier(),
    "EllipseSum": EllipseSumVerifier(),
    # ...
}

def verify(candidate: RelationCandidate) -> VerifyResult:
    v = VERIFIERS[candidate.rel]
    return v.verify(candidate.src_node, candidate.dst_node, candidate.attrs)
```

3. 自适应容差模型 (Tolerance Model)

由于图像存在像素量化与拟合误差，验证必须基于容差。容差采用自适应策略：

$$\text{tol} = \max(\text{absolute_tol}, \text{relative_tol} \times \text{scale})$$

- absolute_tol: 像素级硬下限 (如 2.0 px)。
- relative_tol: 相对尺度 (如 1.5% × 半径 / 线长 / 图像对角线)。
- scale: 取相关对象特征尺寸。

3.1 三态判定规则

设测量误差 $e = |\text{measured} - \text{expected}|$:

$$\text{verified} = \begin{cases} \text{true} & e \leq \text{tol} \\ \text{uncertain} & \text{tol} < e \leq 3 \cdot \text{tol} \\ \text{false} & e > 3 \cdot \text{tol} \end{cases}$$

3.2 各关系容差默认值

关系	absolute_tol	relative_tol	scale
On (点在线)	2.0 px	0.5%	线长
On (点在圆)	2.0 px	1.5%	半径
Perpendicular	3°	—	—
Parallel	3°	—	—
Tangent (直线切圆)	2.0 px	1.5%	半径
Tangent (两圆)	2.0 px	1.5%	(r1+r2)
Equal (线段等长)	—	2%	平均长度
Equal (角相等)	2°	—	—

关系	absolute_tol	relative_tol	scale
EllipseSum	—	1.5%	2a
Collinear	2.0 px	0.5%	包围盒对角线
Concentric	2.0 px	1%	平均半径

3.3 尺度归一化

所有像素距离按 `scale_px_per_cm` (标定) 归一化为厘米, 使容差与图像分辨率无关。 `scale_px_per_cm` 由图像元数据或默认值 (12 px/cm) 给出。

4. 验证示例

4.1 垂直验证 ($OA \perp AB$)

$v1 = OA \text{方向} = (0, -75.5)$
 $v2 = AB \text{方向} = (80, 55.5)$
 $\cos\theta = (v1 \cdot v2) / (|v1| |v2|) = -0.0021$
 $\theta = 90.12^\circ$
 $e = |\theta - 90^\circ| = 0.12^\circ < \text{tol}(3^\circ) \rightarrow \text{verified}=\text{true}$
`evidence = "θ=90.12°, |θ-90°|=0.12°<3°"`

4.2 切线验证 (AB 切圆 O)

$d(0, \text{直线}AB) = |a \cdot 0x + b \cdot 0y + c| / \sqrt{a^2+b^2} = 75.4$
 $r = 75.5$
 $e = |d - r| = 0.1$
 $\text{tol} = \max(2.0, 0.015 \times 75.5) = 2.13$
 $e < \text{tol} \rightarrow \text{verified}=\text{true}$
 切点 = 垂足 = A $\rightarrow \text{attrs.tangent_point} = P_A$
`evidence = "d=75.4, r=75.5, |d-r|=0.1<tol=2.13, tangent\_point=A"`

4.3 椭圆焦点验证 (P 在椭圆上)

$|PF1| + |PF2| = 178.9$
 $2a = 180$
 $e = |178.9 - 180| = 1.1$
 $\text{tol} = 0.015 \times 180 = 2.7$
 $e < \text{tol} \rightarrow \text{verified}=\text{true}$
`evidence = "|PF1|+|PF2|=178.9, 2a=180, e=1.1<tol=2.7"`

4.4 两圆关系验证

$d = |0102| = 120.0$
 $r1=50, r2=70$
 $r1+r2 = 120$
 $e = |d - (r1+r2)| = 0.0 < \text{tol} \rightarrow \text{ExternallyTangent, verified}=\text{true}$

5. 验证日志与可解释性

每次验证记录（断言，证据，容差，结论），作为 `verification_log` 一并输出：

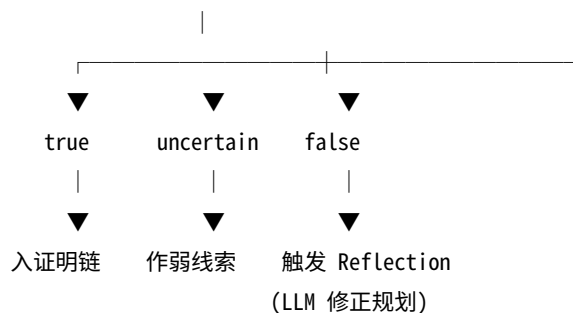
```
{
  "verification_log": [
    {
      "step": "relation", "rel": "Perpendicular", "src": "L_OA", "dst": "L_AB",
      "measured": {"angle": 90.12}, "tol": 3.0, "verified": "true",
      "evidence": "θ=90.12°, |θ-90°|=0.12°<3°",
    },
    {
      "step": "relation", "rel": "Tangent", "src": "L_AB", "dst": "C_0",
      "measured": {"d": 75.4, "r": 75.5}, "tol": 2.13, "verified": "true",
      "evidence": "|d-r|=0.1<tol", "attrs": {"tangent_point": "P_A"}
    }
  ]
}
```

日志使整个系统的判定过程**完全可审计**，是“可解释”原则的直接体现，也便于调试与回归测试。

6. 闭环协议：Verifier ↔ LLM ↔ Solver

Verifier 不仅用于感知层关系验证，还参与推理层闭环：

LLM 提出假设 ——► Verifier.verify(hypothesis)



6.1 工具调用接口

Verifier 作为 LLM 的工具 (tool) 暴露：

```
def verify_tool(relation: str, src: str, dst: str, attrs: dict = {}):
    """LLM 可调：验证一条几何关系。"""
    candidate = RelationCandidate(src=src, dst=dst, rel=relation, attrs=attrs)
    return VERIFIER.verify(candidate).dict()
```

LLM 在证明过程中可主动调用 `verify` 检查其中间结论，形成“假设—验证”闭环。

7. 验证器的正确性保障

Verifier 本身必须是正确的，否则全系统失效。保障措施：

1. **合成数据回归测试**：用程序化合成图（带 ground truth）测试每个验证器，覆盖率 > 95%。
2. **单元测试**：每个验证器针对 true/false/uncertain 三态各有测试用例。
3. **边界测试**：测试容差边界（ $e=tol$ 、 $e=3 \cdot tol$ ）的行为。
4. **交叉验证**：对真题，将 Verifier 结论与人工标注对比，统计准确率。

8. 性能优化

- **批量验证**: 对同类关系批量计算 (向量化)。
 - **空间分桶**: On 类关系用网格分桶, 仅验证邻近对。
 - **缓存**: 对同一 (src,dst,rel) 缓存结果, 避免重复计算。
 - **短路**: false 快速返回 ($e > 3 \cdot \text{tol}$ 即可判 false, 无需精算)。
-

9. 设计路线小结

Constraint Verification Engine 的设计路线为: “**三态输出哲学** → **按关系类型分派验证器** → **自适应容差模型 (绝对 + 相对)** → **证据可读日志** → **与 LLM/Solver 闭环协议** → **合成数据保障正确性**”。它是本系统阻断误差级联、保证数学严格性的核心创新, 使” LLM 提假设、Verifier 判真假” 的分工得以落地。